2.2.2. Các loại kiểm thử (Test Types)

Loại kiểm thử (Test Types) là các nhóm hoạt động kiểm thử (groups of test activities) liên quan đến **đặc tính chất lượng cụ thể (specific quality characteristics)**.

Có rất nhiều loại kiểm thử (test types) tồn tại và có thể được áp dụng trong các dự án. Trong giáo trình này, **bốn loại kiểm thử** sau đây sẽ được đề cập:

1. Functional Testing (Kiểm thử chức năng)
2. Non-functional Testing (Kiểm thử phi chức năng)
3. Black-box Testing (Kiểm thử hộp đen)
4. White-box Testing (Kiểm thử hộp trắng)

Đặc điểm quan trọng:

- ✅ Có thể áp dụng ở **MỌI mức độ kiểm thử (test level)** (Component, Integration, System, Acceptance)
- ✅ Trọng tâm (focus) sẽ **khác nhau ở mỗi level**
  - Component level: Chủ yếu White-box, ít Black-box
  - System/Acceptance level: Chủ yếu Black-box, ít White-box
- ✅ Các kỹ thuật kiểm thử (test techniques) khác nhau có thể được sử dụng cho mỗi loại

1️⃣ Functional Testing (Kiểm thử chức năng)

📘 Định nghĩa

Kiểm thử chức năng (Functional testing): Đánh giá các chức năng (functions) mà một thành phần (component) hoặc hệ thống (system) phải thực hiện. Các chức năng này chính là **"cái gì" (what)** mà đối tượng kiểm thử (test object) nên làm. Mục tiêu chính (main objectives) của kiểm thử chức năng là kiểm tra **tính đầy đủ (completeness)** của chức năng, **tính chính xác (correctness)** của chức năng và **tính phù hợp (appropriateness)** của chức năng.

Trả lời câu hỏi: Hệ thống làm **"CÁI GÌ"** (WHAT)?

🎯 Mục tiêu chính (Main Objectives)

Kiểm tra **3 tính chất** của chức năng:

Tính chất	Tiếng Anh	Giải thích	Ví dụ
Tính đầy đủ	Completeness	Có đủ chức năng theo yêu cầu không?	Requirements có 10 features → Cả 10 đều được triển khai?
Tính chính xác	Correctness	Chức năng hoạt động đúng không?	Login với password đúng → Phải thành công
Tính phù hợp	Appropriateness	Chức năng phù hợp với mục đích không?	Chức năng "Tìm kiếm" có thực sự giúp user tìm được sản phẩm không?

💡 Ví dụ Kiểm thử chức năng (Functional Testing) ở các mức độ (levels):

Component Level:

```
// Test function: calculateShippingFee()
describe('calculateShippingFee - Functional Tests', () => {
  test('Completeness: All order types handled', () => {
    expect(calculateShippingFee(100, 'STANDARD')).toBeDefined();
    expect(calculateShippingFee(100, 'EXPRESS')).toBeDefined();
    expect(calculateShippingFee(100, 'SAME_DAY')).toBeDefined();
  });

  test('Correctness: Returns correct fee', () => {
    expect(calculateShippingFee(100, 'STANDARD')).toBe(30);
    expect(calculateShippingFee(100, 'EXPRESS')).toBe(50);
  });

  test('Appropriateness: Free shipping for orders > 500k', () => {
    expect(calculateShippingFee(600, 'STANDARD')).toBe(0);
  });
});
```

System Level:

```
Test Case: User Login Feature

Completeness:
✓ Login form có đầy đủ fields? (username, password, remember me)
✓ Có "Forgot password" link?
✓ Có "Register" link?

Correctness:
✓ Login với credentials đúng → Success
✓ Login với password sai → Error message
✓ Login với account locked → Locked message

Appropriateness:
✓ Error messages rõ ràng, hữu ích?
✓ "Remember me" thực sự giúp user không cần login lại?
✓ Login flow đủ nhanh? (< 2 giây)
```

Acceptance Level (UAT):

```
Business User Test: E-commerce Checkout

Completeness:
✓ Có đủ các payment methods cần thiết? (COD, Card, E-wallet)
✓ Có đủ shipping options? (Standard, Express)

Correctness:
✓ Tính tổng tiền đúng? (giá + ship - discount)
✓ Apply discount code đúng?
✓ Email confirmation gửi đúng?
```

```
Appropriateness:
✓ Checkout process có dễ hiểu không?
✓ Có quá nhiều bước không? (nên < 5 bước)
✓ Có giúp user hoàn thành mua hàng nhanh không?
```

2️⃣ Non-functional Testing (Kiểm thử phi chức năng)

📘 Định nghĩa

Kiểm thử phi chức năng (Non-functional Testing) đánh giá các thuộc tính (attributes) **khác ngoài đặc tính chức năng (functional characteristics)**.

Trả lời câu hỏi: Hệ thống hoạt động **"TỐT NHƯ THẾ NÀO"** (HOW WELL)?

🎯 Mục tiêu chính (Main Objectives)

Kiểm tra các **đặc tính chất lượng phi chức năng (non-functional quality characteristics)**.

📊 ISO/IEC 25010: 8 đặc tính chất lượng phi chức năng (Non-functional Quality Characteristics)

1. Performance Efficiency (Hiệu quả hiệu năng)

Định nghĩa: Hiệu năng (performance) so với tài nguyên sử dụng (resources used)

Đặc tính con (Sub-characteristics):

- **Time behavior (Hành vi thời gian):** Response time (thời gian phản hồi), processing time (thời gian xử lý)
- **Resource utilization (Sử dụng tài nguyên):** CPU, Memory, Disk usage
- **Capacity (Năng lực):** Maximum users (số người dùng tối đa), throughput (thông lượng)

Ví dụ các bài kiểm thử (test cases):

```
Component Level:
├─ Function calculateTotal() executes in < 100ms
├─ Memory usage < 10MB for 10,000 items
└─ CPU usage < 50% during calculation

System Level:
├─ Homepage loads in < 2 seconds
├─ API responds in < 500ms for 95% of requests
├─ System handles 1000 concurrent users
└─ Database queries execute in < 1 second

Kiểm thử tải (Load Testing):
├─ 1000 users → Response time still < 3s
├─ 5000 users → System still stable
└─ 10000 users → Find breaking point

Kiểm thử căng thẳng (Stress Testing):
├─ Gradually increase load → Find maximum capacity
└─ System degrades gracefully (không crash đột ngột)
```

```
Kiểm thử bền bỉ (Endurance Testing):
├─ Run with 500 users for 24 hours
├─ Check for memory leaks (rò rỉ bộ nhớ)
└─ Performance degradation over time?
```

2. Compatibility (Tính tương thích)

Định nghĩa: Khả năng tương tác (interact) với các hệ thống/môi trường khác

Đặc tính con (Sub-characteristics):

- **Co-existence (Cùng tồn tại):** Hoạt động cùng các phần mềm khác
- **Interoperability (Khả năng tương tác):** Trao đổi thông tin với hệ thống khác

Ví dụ các bài kiểm thử (test cases):

```
Browser Compatibility:
├─ Chrome (latest, -1, -2 versions)
├─ Firefox (latest, -1, -2 versions)
├─ Safari (latest, -1, -2 versions)
└─ Edge (latest, -1 versions)

OS Compatibility:
├─ Windows (10, 11)
├─ macOS (Monterey, Ventura, Sonoma)
├─ Linux (Ubuntu 22.04, 24.04)
└─ Mobile: iOS 16+, Android 12+

Device Compatibility:
├─ Desktop (1920x1080, 1366x768, 1440x900)
├─ Tablet (iPad, Samsung Tab)
└─ Mobile (iPhone, Samsung, Xiaomi...)

Interoperability:
├─ Integration với Facebook Login
├─ Integration với Google Maps API
└─ Import/Export data với Excel
```

3. Usability (Tính khả dụng / Khả năng tương tác)

Định nghĩa: Mức độ dễ sử dụng (ease of use) của hệ thống

Đặc tính con (Sub-characteristics):

- **Learnability (Khả năng học):** Học sử dụng dễ không?
- **Operability (Khả năng vận hành):** Vận hành dễ không?
- **User error protection (Bảo vệ lỗi người dùng):** Ngăn lỗi người dùng
- **Aesthetics (Tính thẩm mỹ):** Giao diện đẹp không?
- **Accessibility (Khả năng tiếp cận):** Tiếp cận cho người khuyết tật

Ví dụ các bài kiểm thử (test cases):

```
Learnability Testing:
├─ New user hoàn thành task đầu tiên trong < 10 phút?
├─ Training material dễ hiểu?
```

```
└─ Onboarding tutorial hiệu quả?

Operability Testing:
├─ Common tasks hoàn thành trong vài click?
├─ Navigation intuitive?
├─ Keyboard shortcuts hữu ích?
└─ Search function hiệu quả?

User Error Protection:
├─ Validation messages rõ ràng?
├─ Confirmation cho critical actions?
├─ Undo/Redo available?
└─ Auto-save để tránh mất dữ liệu?

Aesthetics Testing:
├─ UI design consistent?
├─ Colors, fonts, spacing hợp lý?
└─ Brand identity rõ ràng?

Accessibility Testing:
├─ Screen reader compatible? (cho người khiếm thị)
├─ Keyboard navigation? (cho người không dùng chuột)
├─ Sufficient color contrast? (WCAG 2.1 AA)
├─ Alternative text for images?
└─ Captions for videos? (cho người khiếm thính)
```

4. Reliability (Độ tin cậy)

Định nghĩa: Hệ thống hoạt động ổn định, đáng tin cậy (reliable)

Đặc tính con (Sub-characteristics):

- **Maturity (Mức độ trưởng thành):** Ít lỗi trong điều kiện bình thường
- **Availability (Tính khả dụng):** Uptime cao
- **Fault tolerance (Khả năng chịu lỗi):** Chịu lỗi tốt
- **Recoverability (Khả năng phục hồi):** Phục hồi nhanh sau sự cố

Ví dụ các bài kiểm thử (test cases):

```
Availability Testing:
├─ Target: 99.9% uptime (8.76 hours downtime/year)
├─ Monitor uptime qua 1 tháng
└─ Planned maintenance windows acceptable?

Fault Tolerance Testing:
├─ Database crash → Hệ thống chuyển sang DB dự phòng?
├─ Network timeout → Retry mechanism hoạt động?
├─ Third-party API down → Fallback mechanism?
└─ Partial failure không làm crash toàn bộ hệ thống?

Recoverability Testing:
├─ Database restore time: < 1 hour
├─ Backup frequency: Mỗi 6 giờ
```

```
├─ RTO (Recovery Time Objective): < 4 hours
├─ RPO (Recovery Point Objective): < 1 hour data loss
└─ Disaster recovery plan tested định kỳ?

Maturity Testing:
├─ MTBF (Mean Time Between Failures): > 1000 hours
├─ MTTR (Mean Time To Repair): < 2 hours
└─ Defect density: < 1 defect per 1000 lines of code
```

5. Security (Bảo mật)

Định nghĩa: Bảo vệ thông tin và dữ liệu (protect information and data)

Đặc tính con (Sub-characteristics):

- **Confidentiality (Tính bảo mật):** Bảo mật dữ liệu
- **Integrity (Tính toàn vẹn):** Toàn vẹn dữ liệu
- **Non-repudiation (Không thể chối bỏ):** Không thể chối bỏ
- **Accountability (Trách nhiệm giải trình):** Trách nhiệm rõ ràng
- **Authenticity (Tính xác thực):** Xác thực

Ví dụ các bài kiểm thử (test cases):

```
Confidentiality Testing:
├─ Data at rest encrypted? (AES-256)
├─ Data in transit encrypted? (TLS 1.3)
├─ Sensitive data masked in logs?
└─ Password hashed (bcrypt, Argon2)?

Integrity Testing:
├─ Data không bị modify bởi unauthorized users?
├─ Checksum verification for file uploads?
└─ Database transactions ACID compliant?

Authentication Testing:
├─ Password policy enforced? (min 8 chars, uppercase, number, symbol)
├─ Account lockout after 5 failed attempts?
├─ Multi-factor authentication (MFA) available?
├─ Session timeout after 30 minutes inactivity?
└─ Password reset flow secure?

Authorization Testing:
├─ Role-Based Access Control (RBAC) enforced?
├─ Admin cannot access user's private data?
├─ Horizontal privilege escalation prevented?
├─ Vertical privilege escalation prevented?
└─ API endpoints protected?

Vulnerability Testing:
├─ SQL Injection: ' OR '1'='1 → Prevented?
├─ XSS (Cross-Site Scripting): <script>alert()</script> → Sanitized?
├─ CSRF (Cross-Site Request Forgery): Token verified?
├─ File Upload: .php, .exe files blocked?
```

```
├─ Directory Traversal: ../../../etc/passwd → Prevented?
└─ Security headers present? (CSP, X-Frame-Options, HSTS...)

Penetration Testing:
├─ Ethical hacking attempt
├─ Try to exploit vulnerabilities
└─ Report findings for remediation
```

6. Maintainability (Khả năng bảo trì)

Định nghĩa: Dễ dàng sửa đổi, nâng cấp (easy to modify and upgrade)

Đặc tính con (Sub-characteristics):

- **Modularity (Tính mô-đun):** Chia nhỏ, dễ thay đổi
- **Reusability (Khả năng tái sử dụng):** Tái sử dụng code
- **Analyzability (Khả năng phân tích):** Dễ phân tích
- **Modifiability (Khả năng sửa đổi):** Dễ sửa đổi
- **Testability (Khả năng kiểm thử):** Dễ test

Ví dụ các bài kiểm thử (test cases):

```
Code Quality Metrics:
├─ Code coverage: > 80%
├─ Cyclomatic complexity: < 10 per function
├─ Code duplication: < 5%
├─ Lines of code per file: < 500
└─ Technical debt ratio: < 5%

Modularity Testing:
├─ Loosely coupled modules?
├─ High cohesion within modules?
└─ Dependency injection used?

Documentation Testing:
├─ API documentation complete? (Swagger/OpenAPI)
├─ Code comments adequate?
├─ Architecture diagrams up-to-date?
└─ Deployment guide available?

Testability Testing:
├─ Unit tests easy to write?
├─ Mocking/stubbing supported?
├─ Test data easy to create?
└─ CI/CD pipeline configured?
```

7. Portability (Tính linh hoạt / Khả năng di động)

Định nghĩa: Dễ chuyển đổi môi trường (easy to transfer from one environment to another)

Đặc tính con (Sub-characteristics):

- **Adaptability (Khả năng thích nghi):** Thích nghi môi trường mới
- **Installability (Khả năng cài đặt):** Dễ cài đặt

- **Replaceability (Khả năng thay thế):** Dễ thay thế

Ví dụ các bài kiểm thử (test cases):

```
Adaptability Testing:
├─ Deploy trên AWS, Azure, GCP đều được?
├─ Chạy trên Docker container?
├─ Configuration externalized? (không hardcode)
└─ Database-agnostic? (MySQL, PostgreSQL đều được)

Installability Testing:
├─ Installation wizard dễ hiểu?
├─ Prerequisites clearly documented?
├─ Installation time: < 30 minutes
├─ Uninstall clean (không để rác)?
└─ Upgrade from previous version smooth?

Replaceability Testing:
├─ Migrate từ competitor's product dễ không?
├─ Data import/export đầy đủ?
└─ API compatibility maintained across versions?
```

8. Safety (Tính an toàn)

Định nghĩa: Không gây nguy hại cho con người, môi trường

Đặc biệt quan trọng cho:

- Medical devices (thiết bị y tế)
- Automotive systems (ô tô tự lái)
- Industrial control systems (nhà máy)
- Aviation systems (hàng không)

Ví dụ tests:

```
Medical Device (VD: Máy tim phổi nhân tạo):
├─ Fail-safe mechanisms: Nếu sensor lỗi → Alarm + Switch to safe mode
├─ Redundancy: Backup systems available
├─ Alarms tested: High/low alerts work correctly
└─ Emergency shutdown: < 1 second response time

Automotive (VD: Hệ thống phanh ABS):
├─ Brake failure → Emergency brake activated?
├─ Sensor malfunction → Safe degradation?
└─ Response time: < 100ms for critical functions

Industrial Control System:
├─ Temperature sensor lỗi → Shut down furnace?
├─ Pressure exceed limit → Relief valve opens?
└─ Emergency stop button works always?
```

⏰ Khi nào bắt đầu kiểm thử phi chức năng (Non-functional Testing)?

Nguyên tắc: Bắt đầu **SỚM** trong SDLC (Software Development Life Cycle) là phù hợp!

```
✅ ĐÚNG: Early Non-functional Testing

Component Level:
├─ Performance: Test function execution time
├─ Security: Test input validation
└─ Maintainability: Check code quality metrics

Integration Level:
├─ Performance: Test integration latency
├─ Security: Test API authentication
└─ Compatibility: Test with different databases

System Level:
├─ Performance: Load testing, stress testing
├─ Security: Penetration testing
├─ Usability: User testing
└─ Reliability: Failover testing

❌ SAI: Chỉ test non-functional ở cuối

Risk: Phát hiện muộn các khuyết tật non-functional
→ Mối đe dọa nghiêm trọng cho sự thành công của dự án!
```

🔗 Mối quan hệ giữa Kiểm thử chức năng (Functional) và Kiểm thử phi chức năng (Non-functional)

Nhiều bài kiểm thử phi chức năng (non-functional tests) được dẫn xuất từ các bài kiểm thử chức năng (derived from functional tests):

```javascript
// Functional Test
test('User can login with valid credentials', () => {
  login('user@example.com', 'password123');
  expect(isLoggedIn()).toBe(true);
});

// Non-functional Test (Performance) - Cùng test case, khác tiêu chí
test('Login completes within 2 seconds', () => {
  const startTime = Date.now();
  login('user@example.com', 'password123');
  const endTime = Date.now();

  expect(endTime - startTime).toBeLessThan(2000); // Performance constraint
});

// Non-functional Test (Security) - Cùng test case, khác tiêu chí
test('Login prevents SQL injection', () => {
  const result = login("' OR '1'='1", 'anything');
  expect(result.error).toBe('Invalid credentials'); // Security constraint
});
```

```
// Non-functional Test (Usability) - Cùng test case, khác tiêu chí
test('Login shows clear error message on failure', () => {
  const result = login('user@example.com', 'wrongpassword');
  expect(result.errorMessage).toContain('password is incorrect');
  expect(result.errorMessage).not.toContain('SQL error'); // Usability constraint
});
```

3️⃣ Black-box Testing (Kiểm thử hộp đen)

📘 Định nghĩa

Kiểm thử hộp đen (Black-box Testing) là kiểm thử **dựa trên đặc tả (specification-based)**, không liên quan đến cấu trúc bên trong (internal structure).

Đặc điểm:

- ❌ **KHÔNG** cần biết code bên trong
- ✅ Dựa trên tài liệu (documentation): Requirements, Specifications, User Stories
- 🎯 Mục tiêu (objective): Kiểm tra **hành vi (behavior)** của hệ thống so với **đặc tả (specifications)**

🎯 Khi nào dùng Black-box?

✅ **Phù hợp:**

- System Testing (Kiểm thử hệ thống)
- Acceptance Testing (Kiểm thử chấp nhận)
- Integration Testing (Kiểm thử tích hợp) - góc nhìn giao diện (interface perspective)
- Khi tester không có quyền truy cập code
- Khi muốn test từ góc nhìn người dùng (user perspective)

🛠️ Kỹ thuật kiểm thử hộp đen (Black-box Testing Techniques)

Kỹ thuật	Mô tả	Khi nào dùng
Equivalence Partitioning (Phân vùng tương đương)	Chia input thành các nhóm tương đương (equivalent groups)	Khi có nhiều input values
Boundary Value Analysis (Phân tích giá trị biên)	Test tại biên (boundaries) của các phân vùng	Khi có ranges, limits
Decision Table Testing (Kiểm thử bảng quyết định)	Test tất cả combinations của conditions	Khi có business rules phức tạp
State Transition Testing (Kiểm thử chuyển đổi trạng thái)	Test chuyển đổi trạng thái (state transitions)	Khi hệ thống có states
Use Case Testing (Kiểm thử use case)	Test theo kịch bản người dùng (user scenarios)	Khi có use cases/user stories
Error Guessing (Dự đoán lỗi)	Dự đoán lỗi dựa trên kinh nghiệm (experience-based)	Khi tester có kinh nghiệm

💡 Ví dụ Black-box Testing:

Ví dụ 1: Equivalence Partitioning + Boundary Value Analysis

```
Requirement: Tính phí ship
- Đơn hàng < 200k: Phí 30k
- Đơn hàng 200k-500k: Phí 20k
- Đơn hàng > 500k: Free ship

Black-box Test Design:

Equivalence Partitions:
├─ Partition 1: < 200k
├─ Partition 2: 200k-500k
└─ Partition 3: > 500k

Test Cases (Equivalence Partitioning):
├─ TC1: 100k (Partition 1) → Expected: 30k
├─ TC2: 300k (Partition 2) → Expected: 20k
└─ TC3: 600k (Partition 3) → Expected: 0k

Test Cases (Boundary Value Analysis):
├─ TC4: 199k (just below boundary) → Expected: 30k
├─ TC5: 200k (on boundary) → Expected: 20k
├─ TC6: 201k (just above boundary) → Expected: 20k
├─ TC7: 499k (just below boundary) → Expected: 20k
├─ TC8: 500k (on boundary) → Expected: 20k
└─ TC9: 501k (just above boundary) → Expected: 0k

✓ Không cần biết code tính phí ship như thế nào
✓ Chỉ cần biết requirements
```

Ví dụ 2: Decision Table Testing

```
Requirement: Apply discount
- VIP member + Order > 500k → 20% discount
- VIP member + Order ≤ 500k → 10% discount
- Normal member + Order > 500k → 10% discount
- Normal member + Order ≤ 500k → No discount

Decision Table:

| TC | Member Type | Order Amount | Discount |
|----|-------------|--------------|----------|
| 1  | VIP         | > 500k       | 20%      |
| 2  | VIP         | ≤ 500k       | 10%      |
| 3  | Normal      | > 500k       | 10%      |
| 4  | Normal      | ≤ 500k       | 0%       |

Black-box Test Cases:
├─ TC1: VIP, 600k → Expected: 20% discount
├─ TC2: VIP, 300k → Expected: 10% discount
```

```
├─ TC3: Normal, 600k → Expected: 10% discount
└─ TC4: Normal, 300k → Expected: 0% discount
```

4️⃣ White-box Testing (Kiểm thử hộp trắng)

📘 Định nghĩa

Kiểm thử hộp trắng (White-box Testing) là kiểm thử **dựa trên cấu trúc (structure-based)**, dựa vào việc triển khai bên trong (internal implementation).

Đặc điểm:

- ✅ **CẦN** biết code bên trong
- ✅ Dựa trên: Source code (mã nguồn), Architecture (kiến trúc), Data flow (luồng dữ liệu), Workflow (luồng công việc)
- 🎯 Mục tiêu (objective): **Bao phủ cấu trúc (cover the structure)** ở mức độ chấp nhận được (acceptable level)

🎯 Khi nào dùng White-box?

✅ **Phù hợp:**

- Component Testing (Kiểm thử thành phần) / Unit Testing (Kiểm thử đơn vị)
- Component Integration Testing (Kiểm thử tích hợp thành phần)
- Code review (Xem xét mã nguồn)
- Static analysis (Phân tích tĩnh)
- Khi developer viết tests

🛠️ Các mức độ bao phủ mã nguồn (Code Coverage Levels)

Loại bao phủ (Coverage Type)	Mô tả	Target	Độ khó
Statement Coverage (Bao phủ câu lệnh)	% statements (câu lệnh) được execute (thực thi)	80-90%	Dễ
Branch Coverage (Bao phủ nhánh)	% branches (nhánh) được execute	70-80%	Trung bình
Decision Coverage (Bao phủ quyết định)	% decision outcomes (kết quả quyết định) được test	70-80%	Trung bình
Condition Coverage (Bao phủ điều kiện)	% individual conditions (điều kiện riêng lẻ) được test	60-70%	Khó
Path Coverage (Bao phủ đường đi)	% execution paths (đường thực thi) được test	50-60%	Rất khó
MC/DC (Modified Condition/Decision - Điều kiện/Quyết định được sửa đổi)	Mỗi condition independently affects outcome	40-50%	Cực khó

📚 Tóm tắt quan trọng

🔑 4 Loại kiểm thử (Test Types):

Loại kiểm thử (Test Type)	Trả lời câu hỏi	Mục tiêu chính (Main Objectives)	Mức độ nào (Level)?
Functional (Chức năng)	Làm CÁI GÌ?	Completeness (Tính đầy đủ), Correctness (Tính chính xác), Appropriateness (Tính phù hợp)	Mọi level
Non-functional (Phi chức năng)	Hoạt động TỐT NHƯ THẾ NÀO?	8 đặc tính chất lượng (ISO 25010)	Mọi level, bắt đầu sớm
Black-box (Hộp đen)	Hành vi đúng không?	Verify behavior vs specification (Xác minh hành vi so với đặc tả)	System, Acceptance chủ yếu
White-box (Hộp trắng)	Cấu trúc đủ chưa?	Adequate structure coverage (Bao phủ cấu trúc đầy đủ)	Component, Integration chủ yếu

⚠️ Điểm quan trọng (Key Points):

1. ✅ **Tất cả 4 loại kiểm thử (test types) đều có thể áp dụng ở MỌI mức độ kiểm thử (test level)**
2. ✅ **Trọng tâm (focus) khác nhau ở mỗi level**
3. ✅ **Kiểm thử phi chức năng (Non-functional testing) nên bắt đầu SỚM** (ngay từ component level nếu có thể)
4. ⚠️ **Phát hiện muộn các khuyết tật phi chức năng (non-functional defects) = Mối đe dọa nghiêm trọng (serious threat)**
5. ✅ **Nhiều bài kiểm thử phi chức năng (non-functional tests) được dẫn xuất từ các bài kiểm thử chức năng (derived from functional tests)**
6. ✅ **Các kỹ thuật kiểm thử (test techniques) khác nhau có thể dùng cho mọi test type**

📊 Ma trận áp dụng (Application Matrix):

Loại kiểm thử (Test Type) ↓ / Mức độ (Level) →	Component	Integration	System	Acceptance
Functional (Chức năng)	Logic functions	Interface functions	E2E workflows	Business value
Non-functional (Phi chức năng)	Performance of functions	Integration latency	Load/Stress tests	Usability tests
Black-box (Hộp đen)	Spec-based unit tests	Interface testing	Requirements testing	UAT scenarios
White-box (Hộp trắng)	Code coverage	Integration paths	(Ít dùng)	(Hiếm dùng)

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm

hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.